

PONTEIROS

LINGUAGEM DE PROGRAMAÇÃO

TALITA BERBEL

talita.berbel@fatec.sp.gov.br

Fatec
Sorocaba
José Crespo Gonzales

CP
Centro
Paula Souza

AGENDA

- Introdução
- Declaração
- Inicialização
- Acesso
- Aritmética de Ponteiros
- Ponteiros para acessar Vetores
- Ponteiros e Structs
- Conclusão
- Dúvidas



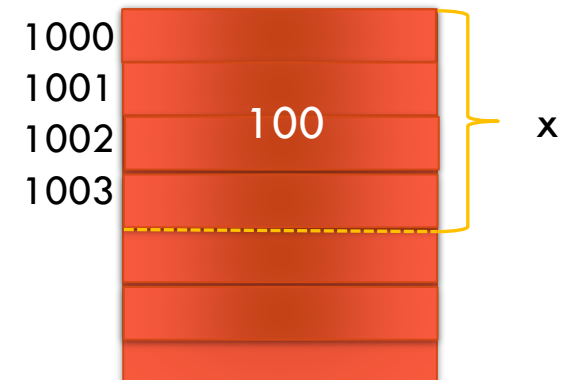
INTRODUÇÃO



- Um **ponteiro** é uma variável especial que armazena o endereço de memória de outra variável. Ele permite acessar e manipular diretamente os dados na memória, tornando os programas mais flexíveis e eficientes.
- Os ponteiros:
- Permitem acessar diretamente **posições de memória**.
- São essenciais em **funções, arrays, strings e structs**.
- Tornam possível o uso de alocação **dinâmica de memória** (malloc, free).
- São fundamentais em linguagens de baixo nível.

INTRODUÇÃO

- O ponteiro proporciona um modo de acesso à variável sem referenciá-la diretamente, e para isso, utiliza-se o **endereço** da variável.
- Quando uma variável é declarada, o compilador reserva uma localização de memória com um endereço **único** para armazená-la. O compilador associa esse endereço ao nome da variável e quando no programa a variável é referenciada, automaticamente acessamos o seu endereço (ou localização correspondente na memória).



INTRODUÇÃO

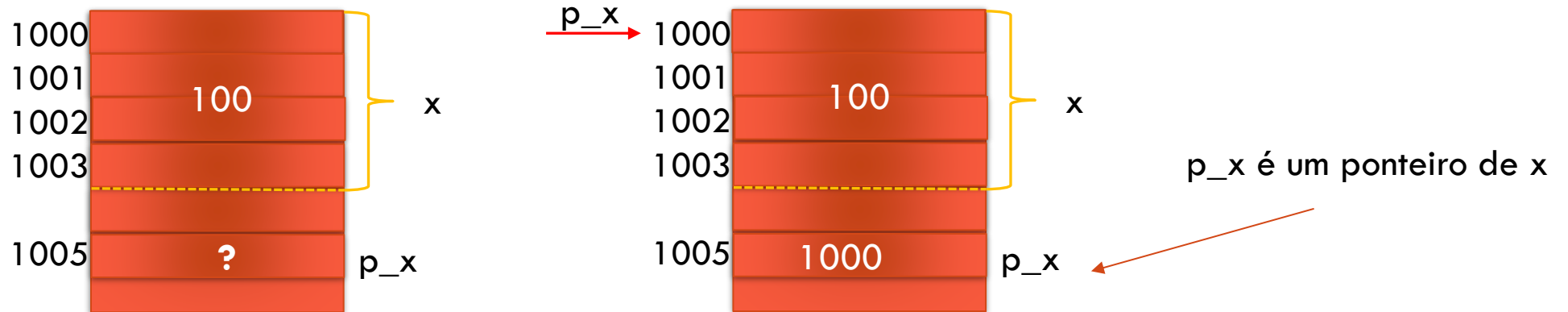
- **Ponteiros constantes** são aqueles que não podem ser alterados e permanecem imutáveis.
- Ponteiro constante é um endereço, que não podem ser alterados, pois passariam a apontar para outro endereço.
- Exemplo:

```
&x; //contém o endereço da variável x
```

```
int lista[10]; //lista contém o endereço inicial do vetor
```

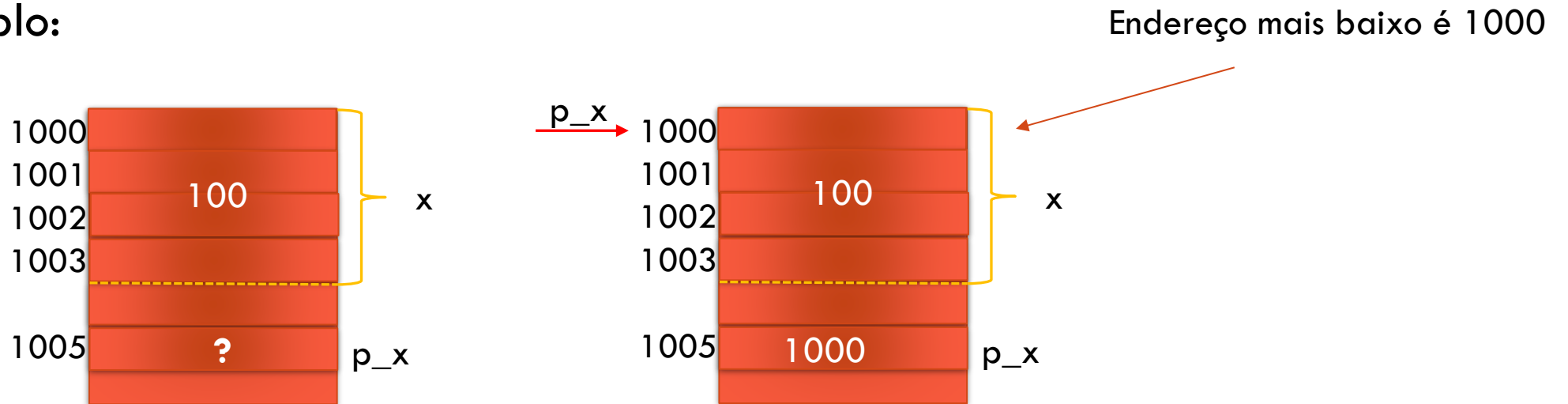
INTRODUÇÃO

- **Ponteiros variáveis** são lugares para guardar endereços.
- Ao criar uma variável `x` e um **ponteiro variável** do tipo da variável `p_x`, deve-se inicializar o `p_x` com o endereço de `x`, logo `p_x` aponta para `x`.
- Exemplo:



INTRODUÇÃO

- Cada tipo de dado ocupa uma quantidade específica de bytes (ex: int = 4 bytes, char = 1 byte). O ponteiro sempre aponta para o endereço **mais baixo** (primeiro byte) da área ocupada pela variável.
- Exemplo:



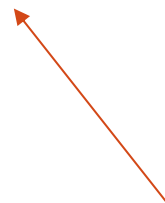
O ponteiro sabe até onde a variável ocupa na memória porque o **tipo declarado** informa **quantos bytes** devem ser acessados a partir do endereço inicial.

DECLARAÇÃO

- Um ponteiro é uma variável numérica e, como todas as variáveis, deve ser declarado antes de ser usado. Ponteiros devem ter o mesmo tipo do dado apontado.
- Utiliza-se o operador “ * ” chamado de **operador de indireção**.
- Sintaxe:
`tipo *nome_ponteiro;`

- Exemplo:

`int *p_x;`



Ponteiro para uma
variável do tipo int

INICIALIZAÇÃO

- Um ponteiro só é útil depois de inicializado, ou seja, após conter um endereço de uma variável.

- Sintaxe:

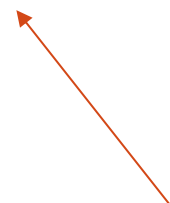
```
nome_ponteiro = &variavel;
```

- Exemplo:

```
int *p_x, x, *p_y, y[10];
```

```
p_x= &x;
```

```
p_y= y;
```



y é o endereço da primeira
posição do vetor

ACESSO

- Após a declaração e inicialização, pode-se utilizar o ponteiro para acessar o conteúdo apontado. O operador `*` precede o nome do ponteiro e é capaz de acessar o conteúdo da variável apontada pelo ponteiro.

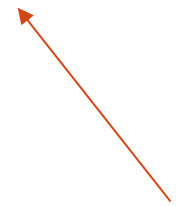
- Sintaxe:

`*nome_ponteiro`

- Exemplo:

```
int *p_x, x=100;  
p_x= &x;
```

```
printf("Valor: %i", *p_x);
```



Acesso indireto

ACESSO

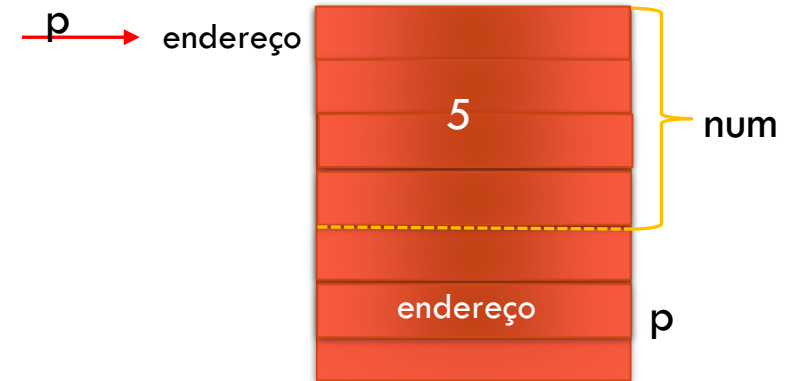
- Exemplo:

```
int main()
{
    int *p, num=5;
    p= &num; //inicialização do ponteiro
```

```
    printf("\nAcesso direto   : %i", num);
    printf("\nAcesso indireto: %i", *p);
```

```
    printf("\nPonteiro constante: %i", &num);
    printf("\nPonteiro variavel  : %i", p);
```

```
}
```



Referem-se ao **conteúdo** de num

Referem-se ao **endereço** de num

PONTEIROS PARA ACESSAR VETORES

- **Exemplo:**

```
int main()
{
    int i, *p, v[5];
    p= v; //inicialização do ponteiro

    for(i=0;i<5;i++)
        scanf("%i",p+i);

    for(i=0;i<5;i++)
        printf("\nValor: %i",*(p+i));
}
```

PONTEIROS PARA ACESSAR VETORES

- **Exemplo:**

```
int main()
{
    int i, *p, v[5];
    p= v; //inicialização do ponteiro

    for(i=0;i<5;i++,p++)
        scanf("%i",p);
    p=v; // reinicializando
    for(i=0;i<5;i++,p++)
        printf("\nValor: %i",*p);
}
```

É necessário para posicionar o ponteiro novamente para o início do vetor

ARITMÉTICA DE PONTEIROS

- Os ponteiros permitem algumas **operações aritméticas**, tais como:
- **Atribuição:**
`p_num = &num // variável simples`
`p_num = num // vetor`
- **Indireção:** `*p_num // conteúdo da variável apontada por p_num`
- **Endereço de:** `&p_num // endereço do ponteiro`
`p_num // endereço da variável apontada`

ARITMÉTICA DE PONTEIROS

- Os ponteiros permitem algumas **operações aritméticas**, tais como:
- **Incremento:** `p_num++` ou `++p_num`
`p_num = p_num + n`
`p_num += n;`
- **Decremento:** `p_num--` ou `--p_num`
`p_num = p_num - n`
`p_num -= n;`
- **Comparação:** válido somente quando os ponteiros apontam para o mesmo vetor.

CONCLUSÃO

- Ponteiros permitem manipular memória diretamente.
- São fundamentais em C.
- Base para vetores, strings e alocação dinâmica.
- E o mais importante: criar programas mais eficientes e dinâmicos compreendendo o funcionamento interno do computador

DÚVIDAS

- Questionamentos?

